



PDF Download
3240765.3240812.pdf
18 January 2026
Total Citations: 8
Total Downloads: 295

 Latest updates: <https://dl.acm.org/doi/10.1145/3240765.3240812>

RESEARCH-ARTICLE

GPU acceleration of RSA is vulnerable to side-channel timing attacks

CHAO LUO

YUNSI FEI

DAVID R. KAEI

Published: 05 November 2018

[Citation in BibTeX format](#)

ICCAD '18: IEEE/ACM INTERNATIONAL
CONFERENCE ON COMPUTER-AIDED
DESIGN

November 5 - 8, 2018
California, San Diego

GPU Acceleration of RSA is Vulnerable to Side-channel Timing Attacks

Chao Luo

luochao@ece.neu.edu
Northeastern University
Boston, MA, USA

Yunsi Fei

yfei@ece.neu.edu
Northeastern University
Boston, MA, USA

David Kaeli

kaeli@ece.neu.edu
Northeastern University
Boston, MA, USA

1 INTRODUCTION

The RSA algorithm [21] is a public-key cipher widely used in digital signatures and Internet protocols, including the Security Socket Layer (SSL) and Transport Layer Security (TLS). RSA entails excessive computational complexity compared with symmetric ciphers. For scenarios where an Internet domain is handling a large number of SSL connections and generating digital signatures for a large number of files, the amount of RSA computation becomes a major performance bottleneck. With the advent of general-purpose GPUs, the performance of RSA has been improved significantly by exploiting parallel computing on a GPU [9, 18, 23, 26], leveraging the Single Instruction Multiple Thread (SIMT) model.

However, the security of RSA on a GPU has not received adequate attention. Side-channel vulnerability (i.e., private key retrieval through side-channel analysis), is particularly critical for cryptosystems. Meanwhile, various side channels, including power, electromagnetic emanation, and timing, have been exploited for implementations of RSA on other computing platforms, such as CPUs, FPGAs, ASICs, and MCUs. Among these attacks, timing attacks have become a realistic cyber threat because they are non-invasive and can be launched remotely. Many timing attacks of RSA implemented on CPUs are micro-architectural (cache-based) attacks [8, 19, 27, 28]. For example, in access-driven attacks including PRIME+PROBE [19] and FLUSH+RELOAD [27], a spy process first sets the cache into a known state, then the victim RSA process runs, and finally the spy measures the timing of its memory accesses to infer the cache access pattern of the victim and finds the secret key. We anticipate applying such cache attacks on a GPU to be very challenging,

if not impossible, due to the high degree of noise resulting from the massive parallel execution on a GPU and the inherently parallel programming model. GPUs use thousands of concurrent threads, so the victim would not be a single process, but a kernel that runs across threads on Streaming Multiprocessors (SMXes). The cache access pattern would be a complex mix of many copies of the same kernel, and therefore the spy cannot easily attribute a particular cache access to a specific victim thread. As different SMXes do not share any cache levels, only when the spy kernel and victim kernel are on the same SMX, there exists a cache side-channel between them. However, the victim kernels may be thread-switched out of the SMX by the scheduler to keep the GPU hardware utilized. This results in a significant amount of noise in the cache access pattern that the spy kernel observes.

An alternative timing attack is time driven - exploiting key-dependent variations in *computation* time, and treating variations in memory accesses as key-independent noise. Such timing attacks heavily depend on the RSA algorithm and associated optimizations. There are two optimizations used in the algorithm: a sliding window exponentiation and the Chinese Remainder Theorem (CRT). A Montgomery multiplication [17] is the basic operation common in many RSA implementations. Kocher [16] first outlined the theoretical foundation for possible timing attacks in several cryptosystems. Dhem *et al.* [6] proposed a practical timing attack of RSA on a smart card in a bit-by-bit fashion. Their attack exploits the variability when using a Montgomery multiplication as the timing leakage, which is also adopted in this work. However, massive concurrent RSA executions on a GPU make this particular timing side channel much more obscure, presenting a challenge for our attack. Our GPU RSA implementation also employs sliding window exponentiation, which was not used by Dhem *et al.* in [6], making the attack based on recovering the exponent, window by window. Tóth proposed another timing attack making the assumption that the total number of extra reductions is known to the adversary [24], which is not a practical attack scenario on a GPU. RSA encryptions using the Chinese Remainder Theorem (CRT), and protocols based on CRT-RSA [1–3], have also been attacked using timing leakage [4, 22]. Our preliminary analysis shows

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD '18, November 5–8, 2018, San Diego, CA, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-5950-4/18/11...\$15.00

<https://doi.org/10.1145/3240765.3240812>

that there may be no difference in the attack strategy for CRT-RSA on a GPU versus a CPU. Therefore, as the first work to investigate side-channel timing leakage of RSA on a GPU, this paper targets an RSA implementation that uses sliding window exponentiation.

There has been very limited prior work focused on timing attacks on a GPU [11, 12]. Our proposed work is distinctively different from the prior work. First, on the timing leakage, our work exploits variations in computation time, while prior work exploited timing differences related to memory accesses, one due to the on-chip coalescing unit [11], and the other due to bank conflicts on an on-chip shared memory [12]. Second, our work targets a different cipher, RSA, while the prior work targets AES implementations. For AES (a block cipher), the data memory address pattern leaks key information. RSA is computationally intensive, and we anticipate such memory-access related timing channels are very weak. In this paper, instead we utilize a computational timing side channel for RSA on a GPU.

When multiple RSA operations are run concurrently with hundreds of threads on a GPU, the interaction between these threads makes it hard to predict their behavior and extract side-channel timing information, reducing the GPU’s vulnerability to attack. The main contributions of this work include: 1) a hierarchical timing model for an RSA implementation on a GPU with Montgomery multiplication and sliding window exponentiation, explicitly capturing various complex interactions in a massively parallel computing platform; 2) a successful correlation timing analysis attack based on the proposed timing model to extract the private key; 3) an effective error correction algorithm designed to detect and correct attack errors to improve the success rate.

The rest of the paper is organized as follows. In Section 2, we briefly introduce the RSA cipher, the optimizations, and the parallel implementation of RSA in CUDA on an NVIDIA GPU [9, 10]. In Section 3, we construct the timing model. Based on the model, a timing attack is proposed in Section 4. In Section 5, we present methods to detect and correct attack errors when they arise. The experimental results are presented in Section 6. Finally, conclusions are drawn in Section 7.

2 BACKGROUND: RSA AND GPU IMPLEMENTATION

The RSA cipher includes a pair of keys, one public and one private. The public key (n, e) is used for encrypting the plaintext message M , and the private key, (n, d) is used for decrypting the ciphertext C :

$$M = C^d \mod n \quad (1)$$

where n is the modulus, e and d are the public and private exponents, respectively. The relationship between n, d, e is that they have to satisfy some constraints to ensure the encryption and decryption are reverse operations of each other [21]. The key length (the bit length of C, M, d , and n) can be 512, 1024, 2048 or more, providing different levels of security. Usually the public key exponent e is chosen to be a relatively small number with a small Hamming weight, while the secret key exponent d is much larger, and requires many more bits. Therefore, the decryption process with the private key is 20 to 60 times slower than the encryption process, demanding acceleration for higher throughput [9].

2.1 Sliding Window Exponentiation

To compute exponentiation of a large number, a simple binary method [15] that performs squaring, and conditional multiplications of each bit of the exponent, is required. One optimization method is to perform sliding-window exponentiation, where the exponent d is decomposed into a series of zero and nonzero windows, F_i of length $L(F_i)$, and the exponentiation is processed window by window. Such sliding window exponentiation can achieve a 21% improvement in performance on average [14].

There are two ways of constructing a sliding window: applying a Variable Length Nonzero Window (VLNW) and using a Constant Length Nonzero Window (CLNW), without much performance difference [20]. In this work, we consider a constant length for the nonzero windows [13], which is 5, 6 and 7 for RSA-512, RSA-1024 and RSA-2048, respectively [9].

The following example illustrates the partitioning of an exponent, with $L = 4$, from right to left:

1001 1101 00000 1011 0111 000 0001 0 0111 000 1011

Note that the exponent starts from a non-zero most significant window, the value of nonzero windows must be odd, and the length of the zero windows varies.

The sliding window exponentiation method for decryption is described in Algorithm 1. There are h windows of exponent d . To begin processing, the odd powers C^w of the method have to be pre-calculated once (Lines 1-4). The computation loop iterates over the most significant window. During each window (iteration), there are a number of squarings (Line 7), which is equal to the bit length of the window, followed by conditional multiplication (only if the window is non-zero) with the multiplicand determined by the window value, F_i (Line 9).

2.2 Montgomery Multiplication

Montgomery’s algorithm [17] is an efficient algorithm for modular multiplication, the basic operation used in the RSA cipher. The multiplier and multiplicand are first transformed

Algorithm 1 Sliding Window

Input: C, d, n (d decomposed into zero and nonzero windows F_i of length $L(F_i)$, for $i = 0, 1, 2, \dots, h-1$)

Output: $M = C^d \bmod n$

```
1:  $C^2 = C \cdot C$ 
2: for  $w = 3, 5, 7, \dots, 2^L - 1$  do
3:    $C^w = C^{w-2} \cdot C^2 \bmod n$ 
4: end for
5:  $M \leftarrow 1$ 
6: for  $i = 0$  to  $h-1$  do
7:    $M \leftarrow M^{2^{L(F_i)}} \quad // \text{squaring}$ 
8:   if  $F_i \neq 0$  then
9:      $M \leftarrow M \cdot C^{F_i} \bmod n \quad // \text{multiplication}$ 
10:  end if
11: end for
12: return  $M$ 
```

into the Montgomerized form before they go through the multiplication. The product is in Montgomerized form as well: $\overline{a \cdot b} = \text{MM}(\overline{a}, \overline{b}) = \overline{a} \cdot \overline{b} \cdot r^{-1} \bmod n$, where r is the power of 2, also co-prime to n and $n < r$.

For decryption, the ciphertext comes in a Montgomerized form. The decryption goes through a series of Montgomery multiplications using the sliding window algorithm discussed in Section 2.1. Inside a Montgomery multiplication, there is a conditional reduction operation dependent on the values of $\overline{a}$ and $\overline{b}$. Because the extra reduction will take more execution time, this data dependency creates a timing side-channel for possible timing attacks. From prior work [22], if $\overline{a}$ and $\overline{b}$ are equal and random, which makes it a squaring operation, the probability of the extra reduction is:

$$\text{Prob}(\text{extra reduction in } \text{MM}(\overline{a}, \overline{a})) = n/3r \quad (2)$$

which is a constant determined by n and r only. If $\overline{a}$ is random, and $\overline{b}$ is a constant, the probability becomes:

$$\text{Prob}(\text{extra reduction in } \text{MM}(\overline{a}, \overline{b})) = \overline{b}/2r \quad (3)$$

i.e., dependent on $\overline{b}$ and r .

2.3 GPU Parallelization of RSA

NVIDIA GPU hardware features many Streaming Multiprocessors (SMXes), with each consisting of homogeneous cores and shared resources such as caches. In GPU programming, the workload is partitioned into a grid of blocks, and each block consists of many threads. Threads from one block will only be executed on the same SMX. On CUDA cores, threads of one block are grouped into warps of 32 threads, executing instructions in a synchronized fashion. Whenever the execution path of threads in a warp diverge, only one path is executed at a time and the others have to wait. The

divergence makes the program run slower, and causes data-dependent time variations. So the execution time is related with the message contents and the private key. Warps on the same SMX are competing for the limited hardware resources, such as CUDA cores, warp schedulers and the shared memory. For warps on different SMXes, they are mostly running independently.

We adopt the parallel RSA implementation in CUDA described by Jang et al. [9, 10]. Note that in the *for* loop iterations in Algorithm 1, when used for processing the exponent windows, they are run in parallel. This is because each iteration depends on the results of previous iteration. Any parallelization is accomplished between messages and within messages. When processing a batch of ciphertext messages on a GPU, the messages are independent and decrypted in parallel, while using the same secret key exponent. In addition, the Montgomery multiplication inside one message's computations is also parallelized. Each message is represented by an array of s 64-bit words, and the multiplication is carried out by s threads, executing in cooperation. For example, for decryption of one message in RSA-512, the s value is 8, i.e., it requires 8 threads to decrypt. A batch of 32 messages needs 256 threads in total, which are divided into 2 blocks, with a typical block size of 128 threads.

3 THE TIMING MODELS OF RSA ON GPUS

The parallelization of workload on a GPU makes the prediction of the execution time much harder than predicting sequential computing time on a CPU. Program execution includes both concurrent computation and blocking execution (e.g., for divergence in a warp). The execution time is not simply equal to the addition of all instructions any more.

In this section, we introduce a hierarchical timing model for RSA decryption on a GPU, which accounts for various interactions during execution, and is the basis for enabling our side-channel timing attack. Note that we choose the extra reduction in the Montgomery multiplication as the timing channel. The number of reductions depends on both the input message and each window of the secret exponent d . We start with a general linear timing model as follows:

$$T = O + v \cdot R \quad (4)$$

where R is the number of extra reductions in the Montgomery multiplications, v is the unit execution time of one reduction, and O represents other key-independent execution time which follows a distribution when the input message varies.

For a sequential implementation, R will simply be the total number of extra reductions performed in each window. With multiple messages being decrypted in parallel by many threads on the GPU, there are complex interactions between the threads through shared resources and the specific GPU programming model. We build a hierarchical model of R

according to the interaction at different levels of parallelism, from a fine-grained thread (message) level, to the warp level, the SM level, and finally for the entire GPU.

3.1 GPU Timing Model

We begin by considering only one message being decrypted by multiple threads (e.g., 16 for RSA-1024, which are within the same warp). Since threads in a warp are synchronized, we can still view the decryption for one message as sequential execution. For each window of d , we calculate the reduction number, as shown in lines 7-10 in Algorithm 1. The first *for* loop (lines 2-4) is ignored here because it is independent of the secret key d .

For the i^{th} window F_i of d , the intermediate variable M is first squared $L(F_i)$ times. If F_i is nonzero, its length $L(F_i)$ is always L when using the CLNW method. When F_i is zero, its length can vary from 1 to the key length. Under the assumption that each bit of d is independent and uniformly distributed between 0 and 1, the longer the length of F_i , the smaller the probability will be. We assume the length of the nonzero window is no longer than 32 bits (probability of 2.33×10^{-10}). We use $r_{msg}(i)$ to record the number of reductions of the squarings for window i . The reduction number of the whole decryption of message, R_{msg} , is the sum of $r_{msg}(i)$ over i .

Note we do not include the extra reduction of the multiplication operation (line 9 of Algorithm 1). Given that the probability of executing the reduction depends on the value of C^{F_i} , as expressed in Equation (3), there may be other windows with the same value of F_i . This kind of dependency on an extra reduction in multiple windows will bias the attack result [6], and an effective attack should not use the reduction of the multiplication.

Next, we consider running a warp using multiple messages in parallel. The number of messages in a warp depends on the key size. There are 2 messages in a warp for RSA-1024 and 1 for RSA-2048. Because threads in one warp are synchronized, the operations on different messages are also synchronized, (i.e., all the messages are processing the same window of size d at the same time). For one squaring operation in a window, if one or more messages have an extra reduction, the whole warp has to wait for it to finish, and the warp is considered as having an extra reduction. Thus, the number of reductions in a warp for a window $r_{warp}(i)$ is not simply the sum of the reductions used for squaring for all the messages.

To calculate $r_{warp}(i)$, we use a binary number, $b_{msg}(i, j)$, to record the extra reductions during execution, for the j^{th} message in the i^{th} exponent window. The length of $b_{msg}(i, j)$ is $L(F_i)$, because in each window there are number of $L(F_i)$ squaring operations, and each bit of $b_{msg}(i, j)$ is for the corresponding squaring. If the squaring of a message results in

an extra reduction, this bit is 1; otherwise, it is 0. For each squaring, all the messages in a warp are synchronized, (i.e., as long as there is one message resulting in an extra reduction, the warp (all the messages) encounters a delay). Therefore, all $b_{msg}(i, j)$ should have bit-wise OR operations over j , and the number of reductions in the warp, $r_{warp}(i)$, is the Hamming weight of such bit-wise OR result of all $b_{msg}(i, j)$ over j . The reduction number of the warp for the entire decryption R_{warp} is also the sum of $r_{warp}(i)$ over i .

For one SMX with multiple warps running, there is no synchronization between warps. When the total thread number (warp number $\times 32$) is equal to or smaller than the number of CUDA cores on the SMX, these warps will run mostly in parallel. However, the reduction operation involves shared memory accesses, which will be processed in a serialized manner, and therefore all the warps are processed in a blocking way to some extent. In scenarios where the total thread number is much larger than the number of SMX CUDA cores, the warps have to be scheduled in series. Based on these considerations, we let the reduction number of one SMX, R_{smx} , to be approximately equal to the sum of its warps' reductions.

For the complete decryption with multiple SMXes, we assume the SMXes are running independently, and the time of one decryption is determined by the slowest SMX. The reduction number of one decryption on the GPU is R_{gpu} , which is the maximum of all the SMX reductions, R_{smx} .

3.2 Timing Model Verification

To verify the timing model, we run multiple experiments of RSA-1024 on an Nvidia K40 GPU. We set the block size to 128, such that each block will have 4 warps. We run experiments and vary the message number for one decryption from 1, 2, 8 to 64. We will have 1 message, 1 warp, 1 block (on one SMX) and 8 blocks (on 8 SMXes), respectively. For each experiment, we run the decryption 10K times with random messages, record the execution time of each decryption using the CPU's clock, and calculate the reduction number for the workload according to the timing model.

Our results are shown in Figure 1, with the execution time normalized, and correlation coefficients (ρ) shown at the top. The figure shows higher linear correlation between the measured execution time and calculated reduction number for (a) and (b), when we have one message and one warp running, respectively. The points in (c) and (d) spread out more for multiple warps and multiple SMXes, due to the approximation of the reduction number for one SMX.

4 CORRELATION TIMING ATTACK

In this section, we propose an attack method for the RSA GPU implementation, based on the timing models we built in Section 3. We assume the ciphertext message is random,

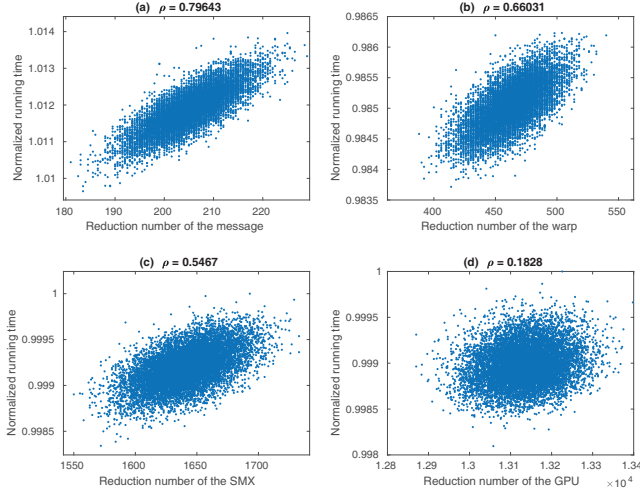


Figure 1: Timing model verification

but known to the attacker. For each decryption run, the adversary only records the total execution time.

The secret key d is extracted one window after another starting from the most significant bit, following the same order as decryption, as shown in Algorithm 1. For the i^{th} target window, the previous attack results are used to calculate the value of the intermediate variable M at the beginning of i^{th} iteration. To avoid confusion with M in other steps in the algorithm, we denote it as M_{temp} . For the first window, M_{temp} is 1.

In the attack, we make a guess of the target window. Based on the guess, we calculate the corresponding reduction number. We compute the Pearson correlation coefficient between the reduction number and the observed timing information. To determine the correct value for a nonzero window or length for a zero window, we iterate over all possible guesses for the target window, and pick the one with the largest correlation as our attack result.

To calculate the reduction number of the i^{th} window for one message, we assume there are at least two windows after the target window. Because the window could be zero or nonzero, we divide our guess into two groups, the length of zero windows and values of non-zero windows, different from the previous work [6] which only works bit by bit. If the window is a zero, its length, L_x , is varying, and there will be number of L_x squarings. For the sliding window algorithm, a zero window is always followed by a nonzero window, while there may be multiple nonzero windows in a sequence. Therefore the next window must be nonzero of length L . There will be another L squarings in the next iteration, without a multiplication in between due to the zero window. From another perspective, the operations can

also be considered as L squaring operations, followed by L_x squaring operations.

If the window is a nonzero window, we guess its value as V_x (V_x is odd). The following operations on M_{temp} are L squaring operations and one multiplication by C^{V_x} . Beyond the multiplication, we can still infer the operations by checking the next one or two windows. If the next window is nonzero, the operation beyond multiplication is another L squarings. If the next window is zero, the window after the next window is nonzero, and we have more than L squaring operations, as discussed in the first situation.

In conclusion, the first group of operations on M_{temp} are L squaring operations. After that, if the window is zero, the subsequent operations are L_x squarings; if the window is nonzero with V_x value, the subsequent operations are multiplication by C^{V_x} and L squarings, as shown in Figure 2.

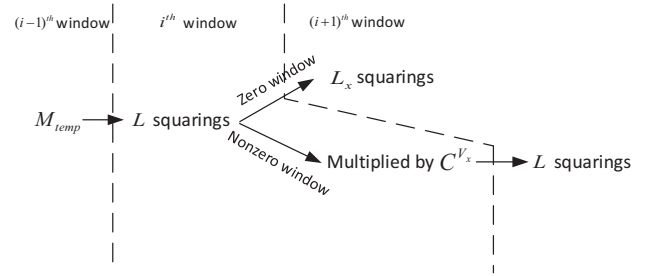


Figure 2: Operations on M_{temp}

When calculating the reduction number, we exclude the first L squaring operations because they are common to all guesses. For a zero window, we record the extra reductions of the next L_x squarings in $r_{zero}(L_x, i, j)$ for each message, where i is the window index, and j is the message index. For a nonzero window, we record the extra reductions for the next L squaring, in $r_{nonzero}(V_x, i, j)$, where the value V_x is used in computing the multiplication C^{V_x} .

We then subsequently calculate the reduction numbers for warps, SMXes, and the GPU for the target window according to the timing model described in Section 3. Note that in our attack model, we are attacking window by window, and the calculated reduction number is only for the current window. While in Section 3.2, the linear correlations are for the sum of the reduction numbers of all the windows. In our attack, we utilize the small linear correlation of one window, which will have a lower correlation coefficient than the one for all h windows shown in Figure 1.

5 ERROR CORRECTION

The inherent drawback of the proposed attack, due to the iterative processing shown in Section 4, is that the attack on the target window is highly dependent on previous attack

results. Therefore, if any of the previous guesses are wrong, the current attack may not succeed because the calculation M_{temp} and reduction number will be wrong. As a result, the correlation coefficients after the first error will be much lower than those before, as show in Figure 3. Each point in Figure 3 is the maximum correlation coefficient of one window attack when we iterate over all possible values. This feature can be utilized to detect errors if there exists N consecutive window attacks with low correlation coefficients.

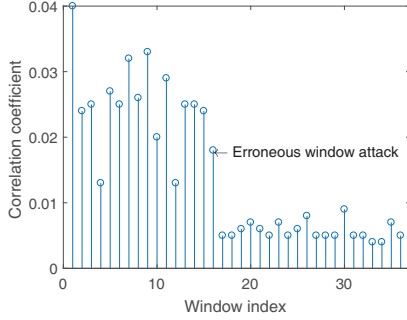


Figure 3: Sequence of correlation coefficients of a timing attack when an error happens

In our attack, the length of a zero window is much harder to attack. Because the number of extra reductions is roughly linear with the bit length of the zero window, the distinguishability provided by the correlation attack is fairly small. For example, the correlations of wrong length guesses are very close to that of a correct one, as show in Figure 4 (a) for the case of attacking a zero window, where the length of the window is 4. If a wrong guess ends up with the highest correlation, the error can only be detected in later windows. We define a parameter, *distinguishability* D , to indicate how likely an error can happen. The value of D for one window is defined as the normalized difference between the largest correlation and the second largest one. The lower D is, the more likely the error will happen. Based on these observations, we propose two error correction methods, forward and backward.

Forward error correction is triggered when the distinguishability D of a target window is below a threshold D_{th} . We need to reconsider the guessed values which do not end up with the highest correlation. We proceed with tentative attacks on the next window multiple times, by setting the target window’s value to other possible values, one after another. For each of those tentative attacks on the next window, we get a maximum correlation coefficient. To pick the right value for the target window, we choose the one resulting in the largest maximum correlation in the next window.

Backward error correction is triggered when there are N consecutive window attacks, with the correlation coefficients

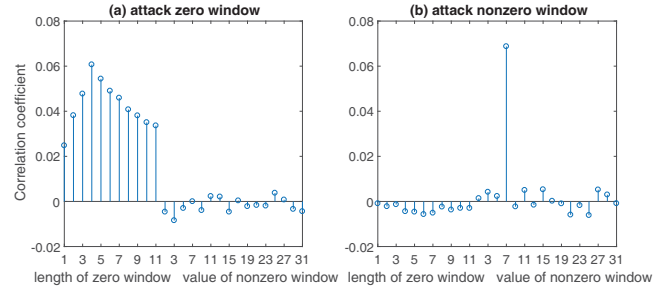


Figure 4: Correlation Coefficients of attacking zero and nonzero windows

below C_{th} . The attack rolls back to the first window of the N consecutive windows. This time, we choose the value for this window with a second-maximum correlation, and then proceed. If this still results in an error and the window is attacked again, the next ranked guess should be chosen to proceed. If all possible values for the window have been tried and there is still an error, we move further backward to the previous window and reconsider other guessed values.

Parameter selection: In our error detection method, there are three parameters, D_{th} , N and C_{th} . Their values affect the effectiveness and convergence speed of the attack. For D_{th} , if it is set too high, the forward error correction will be launched too often (a false positive) and the attack will be slowed down. If it is too low, the chance of missing the detection of an error is high (a false negative). The error is only possibly detected by the backward error correction, which is much slower than the forward one. So a suitable D_{th} should balance the speed of the attack and the error detection rate. Similarly, both error detection rate and attack speed will be affected by the value of N . The choice of C_{th} has a similar impact as N , but in the opposite direction. This means a larger C_{th} is equal to choosing a smaller N . In our experiments, these values are chosen to maximize the success rate, with the attack speed in an acceptable range.

6 EXPERIMENTAL RESULTS AND COUNTERMEASURES

We implement the RSA cipher on an Nvidia K40 GPU, which has 15 SMXes and 192 CUDA cores on each SMX, hosted on a workstation running Ubuntu 14.04.05 LTS with an Intel E5-1603 processor. We record the timing information using the CPU clock which runs at 2.8GHz, instead of the more accurate GPU clock, because normally it is not accessible to the attacker.

We run experiments with different key lengths of 512, 1024 and 2048. The number of threads launched in one decryption varies from 32 to 1024. For each experiment, 100K decryption runs are carried out with random input messages. The error correction parameters D_{th} , N and C_{th} are set to 0.2, 8 and

0.01, respectively. The time used for one attack ranges from around 15 minutes for RAS-512 with 32 threads, to 8 hours for RSA-2048 with 1024 threads. The larger the key size and the number of threads, the higher the attacking time because of computation complexity and more error corrections needed.

Table 1 shows the number of errors corrected by forward and backward error correction, and how many (least significant) windows are incorrect after the attack. In the results, we see the number errors increases as the number of threads and the key size increase. The forward error correction works best with fewer threads and a smaller key size. The backward error correction works best for more threads and a larger key size. The errors left in our experiment are very small (0-3 windows, 18 bits at most), and the full key can be easily recovered using a lattice attack [5] or brute force attack.

Table 1: Attack result with error correction

Threads#	32			512			1024		
	FW	BW	E	FW	BW	E	FW	BW	E
RSA-512	0	0	1	4	2	3	3	2	1
RSA-1024	7	2	2	10	5	1	5	19	3
RSA-2048	12	1	0	7	16	0	24	13	0

FW/BW: forward/backward error corrections; E: errors left

Common countermeasures for sequential RSA implementations on a CPU, which eliminate the data dependent variation of the reductions [7, 25], or hide the messages and exponent with random numbers [16], are also applicable to GPU implementations. However, they will introduce a significant performance degradation, which is a major concern for GPU acceleration. We evaluate an always-reduce countermeasure on GPU, which increases the execution time by 4.5%. We also compare the Pearson correlation coefficient between the number of reductions and the execution time for one message, and show the result in Figure 5. The correlation reduces significantly from 0.79 to 0.06, rendering the time leakage very small and hard to exploit any more.

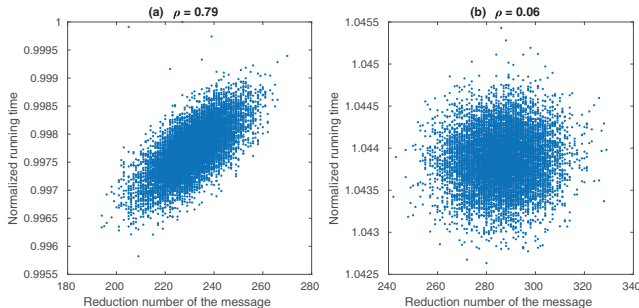


Figure 5: Always reduce countermeasure

Special countermeasures targeting GPU implementations can also be devised. For example, we can randomize the assignment of messages to threads and blocks such that the execution time cannot be correctly predicted by the attacker. We can also add more noise to the timing measurements with dummy random messages, at the cost of performance and throughput.

7 CONCLUSIONS

In this work, we evaluate the vulnerability of an RSA GPU implementation to side-channel timing analysis. We build timing models for a sliding-window RSA implementation with Montgomery multiplications, considering a GPU’s parallel execution characteristics. The attack methods are designed based on the timing model. We obtain attack results from GPU K40 hardware, using different key sizes and working across different levels of parallelism. Our results show that an RSA implementation on a GPU is vulnerable to side-channel timing attacks, calling for both general countermeasures and GPU-specific countermeasures.

ACKNOWLEDGMENTS

This work is supported in part by the National Science Foundation under grants STARSS-1618379 and MRI-1337854.

REFERENCES

- [1] Onur Aciicmez, Werner Schindler, and Çetin K. Koç. 2005. Improving Brumley and Boneh Timing Attack on Unprotected SSL Implementations. In *Proc. ACM Conf. on Computer & Communications Security*. ACM, New York, NY, USA, 8.
- [2] Cyril Arnaud and Pierre-Alain Fouque. 2013. Timing attack against protected RSA-CRT implementation used in PolarSSL. In *Cryptographers Track at the RSA Conference*. Springer, 18–33.
- [3] Billy Bob Brumley and Nicola Tuveri. 2011. Remote Timing Attacks Are Still Practical. In *ESORICS 2011*.
- [4] CaiSen Chen, Tao Wang, and Junjian Tian. 2013. Improving timing attack on RSA-CRT via error detection and correction strategy. *Information Sciences* (2013).
- [5] Don Coppersmith. 1997. Small Solutions to Polynomial Equations, and Low Exponent RSA Vulnerabilities. *Journal of Cryptology* 10, 4 (1997), 233–260.
- [6] Jean-François Dhem, François Koeune, Philippe-Alexandre Leroux, Patrick Mestré, Jean-Jacques Quisquater, and Jean-Louis Willems. 2000. A Practical Implementation of the Timing Attack. In *CARDIS’98*.
- [7] Gaël Hachez and Jean-Jacques Quisquater. 2000. Montgomery Exponentiation with no Final Subtractions: Improved Results. In *CHES 2000*.
- [8] Mehmet Sinan Inci, Berk Gülmezoglu, Gorka Irazoqui Apecechea, Thomas Eisenbarth, and Berk Sunar. 2015. Seriously, get off my cloud! Cross-VM RSA Key Recovery in a Public Cloud. *IACR Cryptology ePrint Archive* 2015 (2015), 898.
- [9] Keon Jang, Sangjin Han, Seungyeop Han, Sue Moon, and KyoungSoo Park. 2011. SSLShader: Cheap SSL Acceleration with Commodity Processors. In *NSDI’11*. USENIX Association, Berkeley, CA, USA, 1–14.

- [10] Keon Jang, Sangjin Han, Seungyeop Han, and KyoungSoo Park. 2015. libgpcrypto. <https://github.com/lwakefield/libgpcrypto>
- [11] Z. H. Jiang, Y. Fei, and D. Kaeli. 2016. A complete key recovery timing attack on a GPU. In *HPCA*. 394–405.
- [12] Zhen Hang Jiang, Yungsi Fei, and David Kaeli. 2017. A Novel Side-Channel Timing Attack on GPUs. In *GLSVLSI '17*. ACM, New York, NY, USA, 167–172.
- [13] G Knuth. 1998. The Art of Computer Programming, Seminumerical Algorithms, Vol. 2, Addition Wesley. Reading, Massachusetts (1998).
- [14] Cetin K Koç. 1995. Analysis of sliding window techniques for exponentiation. *Computers & Mathematics with Applications* 30, 10 (1995), 17–24.
- [15] Cetin Kaya Koç. 1994. *High-speed RSA implementation*. Technical Report. Technical Report, RSA Laboratories.
- [16] Paul C. Kocher. 1996. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In *CRYPTO '96*.
- [17] Peter L Montgomery. 1985. Modular multiplication without trial division. *Mathematics of computation* 44, 170 (1985), 519–521.
- [18] Andrew Moss, Daniel Page, and Nigel P. Smart. 2007. Toward Acceleration of RSA Using 3D Graphics Hardware. In *Cryptography and Coding*.
- [19] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache Attacks and Countermeasures: The Case of AES. In *CT-RSA*.
- [20] H Park, K Park, and Y Cho. 1999. Analysis of the variable length nonzero window method for exponentiation. *Computers & Mathematics with applications* (1999).
- [21] R. L. Rivest, A. Shamir, and L. Adleman. 1978. A Method for Obtaining Digital Signatures and Public-key Cryptosystems. *Commun. ACM* 21, 2 (Feb. 1978), 7.
- [22] Werner Schindler. 2000. A Timing Attack against RSA with the Chinese Remainder Theorem. In *CHES 2000*.
- [23] Robert Szerwinski and Tim Güneysu. 2008. Exploiting the Power of GPUs for Asymmetric Cryptography. In *CHES 2008*.
- [24] R. Tóth, Z. Faigl, M. Szalay, and S. Imre. 2008. An advanced timing attack scheme on RSA. In *Int. Telecommunications Network Strategy and Planning Symposium*, Vol. Supplement. 1–9.
- [25] C. D. Walter. 1999. Montgomery exponentiation needs no final subtractions. *Electronics Letters* 35, 21 (Oct 1999), 1831–1832.
- [26] Yang Yang, Zhi Guan, Huiping Sun, and Zhong Chen. 2015. Accelerating RSA with Fine-Grained Parallelism Using GPU. In *ISPEC*.
- [27] Yuval Yarom and Katrina Falkner. 2014. FLUSH+ RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. In *USENIX Security*, Vol. 2014.
- [28] Yuval Yarom, Daniel Genkin, and Nadia Heninger. 2016. CacheBleed: A Timing Attack on OpenSSL Constant Time RSA. In *CHES 2016*.